

Programación Orientada a Objetos



2025

Objetivos: Juego de Dados

— — —

Reglas del Juego

- Se juega 10 rondas, en **cada** ronda los jugadores tiran **dos** dado.
- El jugador que saca **más de 7** y **le gana al otro jugador** suma un punto.
- Gana el Jugador que más puntos sumó.

Implementación

— — —

```
public class Juego {  
  
    private int puntos1;  
    private int puntos2;  
  
    public void jugar() {  
        int tirada1, tirada2;  
        for (int i =0 ; i<10; i++) {  
            tirada1 = tirarDado() + tirarDado();  
            tirada2 = tirarDado() + tirarDado();  
            if (tirada1>tirada2 && tirada1>=7) {  
                puntos1++;  
            } else {  
                if (tirada2>tirada1 && tirada2>=7) {  
                    puntos2++;  
                }  
            }  
        }  
        String ganador = ganador();  
        if (ganador!= null) {  
            System.out.println("Gano el " + ganador);  
        } else {  
            System.out.println("Empate");  
        }  
    }  
}
```

Implementación

```
public int tirarDado() {  
    return (int) (Math.random()*6 +1);  
}
```

```
public String ganador() {  
    if (puntos1>puntos2) {  
        return "Jugador 1";  
    } else {  
        if (puntos2>puntos1) {  
            return "Jugador 2";  
        }  
        else {  
            return null;  
        }  
    }  
}
```

```
public Juego() {  
    puntos1=0;  
    puntos2=0;  
}
```

```
public static void main(String[] args) {  
    Juego jj = new Juego();  
    jj.jugar();  
}
```

Análisis de la solución

- Todo centralizado
 - Prácticamente una solución procedural
- No se identificaron correctamente los objetos. El modelo dista de la realidad.
- No se distribuyeron correctamente las responsabilidades.
- Difícil de mantener, modificar, extender.



¿Es una solución
Orientada a
Objetos?

Un aspecto importante es determinar “qué es un buen diseño y porque”

Análisis del problema

Identificar los objetos del juego

Identificar las responsabilidades de los objetos de este juego

¿ Qué objetos deben colaborar entre sí ?

Importante: este es un paso creativo con una variedad de soluciones

Dado



¿Que tiene un dado?

Estado

¿Que sabe hacer un dado?

Comportamiento

Cantidad de Caras

Último valor que salio

Preguntar cantidad de caras

~~Cambiar cantidad de caras~~

Pedirle el último valor

~~Cambiar el último valor~~

Tirarse (es responsable de
saber como se tira)

Dado



```
public class Dado {  
    private int cantCaras;  
    private int ultimoValor;  
  
    public int getCantCaras() {  
        return cantCaras;  
    }  
  
    public int getUltimoValor() {  
        return ultimoValor;  
    }  
  
    public int tirar() {  
        ultimoValor = (int) (Math.random()*getCantCaras() +1);  
        return ultimoValor;  
    }  
}
```

```
public Dado() {  
    this(6);  
}  
public Dado(int caras) {  
    cantCaras=caras;  
    this.tirar();  
}
```

Clase Juego

```
public class Juego2 {  
    private int puntos1;  
    private int puntos2;  
    private Dado d1;  
    private Dado d2;
```

Se agregan los
datos

Se delega al
dado el tirar

```
    public void jugar() {  
        int tirada1, tirada2;  
  
        for (int i = 0 ; i < 10; i++) {  
            tirada1 = d1.tirar() + d2.tirar();  
            tirada2 = d1.tirar() + d2.tirar();  
  
            if (tirada1 > tirada2 && tirada1 >= 7) {  
                puntos1++;  
            } else {  
                if (tirada2 > tirada1 && tirada2 >= 7) {  
                    puntos2++;  
                }  
            }  
        }  
        String ganador = ganador();  
        if (ganador != null) {  
            System.out.println("Gano el " + ganador);  
        } else {  
            System.out.println("Empate");  
        }  
    }  
}
```

```
    public String ganador() {  
        if (puntos1 > puntos2) {  
            return "Jugador 1";  
        } else {  
            if (puntos2 > puntos1) {  
                return "Jugador 2";  
            }  
            else {  
                return null;  
            }  
        }  
    }  
}
```

No cambia

Clase Juego

```
public class Juego2 {  
    private int puntos1;  
    private int puntos2;  
    private Dado d1;  
    private Dado d2;
```

Se agregan los dados

```
    public Juego2() {  
        puntos1=0;  
        puntos2=0;  
        d1 = new Dado();  
        d2 = new Dado();
```

```
    public Juego2(Dado d1, Dado d2) {  
        puntos1=0;  
        puntos2=0;  
        this.d1 = d1;  
        this.d2 = d2;
```

```
        return null;
```

```
    } else {  
        System.out.println("Empate");  
    }
```

Hay que crearlos en el juego o que nos den los dados

```
    public static void main(String[] args) {  
        if (args.length > 0) {  
            Dado d1 = new Dado(args[0]);  
            Dado d2 = new Dado(args[1]);  
            Juego2 juego = new Juego2(d1, d2);  
            juego.jugar();  
        } else {  
            Dado d1 = new Dado();  
            Dado d2 = new Dado();  
            Juego2 juego = new Juego2();  
            juego.jugar();  
        }
```

No cambia

Jugador

Sabe su nombre y
posee los puntos
del juego

¿Qué otra
responsabilidad
tiene?



```
public class Jugador {
```

```
    private String nombre;  
    private int puntos;
```

```
    public Jugador(String ss) {  
        nombre = ss;  
        puntos = 0;  
    }
```

```
    public String getNombre() {  
        return nombre;  
    }
```

```
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }
```

```
    public int getPuntos() {  
        return puntos;  
    }
```

```
    public void sumarPunto() {  
        puntos ++;  
    }
```

Jugador

Sabe su nombre y

posee 100 puntos

del juego

responsable de
tiene?

En el juego es el jugador el que
tira los dados

```
public class Jugador {
```

```
    private String nombre;  
    private int puntos;
```

```
    public Jugador(String ss) {  
        nombre = ss;  
        puntos = 0;
```

```
    public int tirarDados(Dado dadito1, Dado dadito2 ) {  
        int valor= dadito1.tirar()+dadito2.tirar();  
        return valor;
```

```
    nombre) {
```

```
    }  
    public void sumarPunto() {  
        puntos ++;  
    }
```



Clase Juego

```
public class Juego3 {  
    private Jugador j1;  
    private Jugador j2;  
    private Dado d1;  
    private Dado d2;
```

```
public void jugar() {  
    int tirada1, tirada2;  
    for (int i = 0 ; i < 10; i++) {  
        tirada1 = j1.tirarDados(d1, d2);  
        tirada2 = j2.tirarDados(d1, d2);  
  
        if (tirada1 > tirada2 && tirada1 >= 7) {  
            j1.sumarPunto();  
        } else {  
            if (tirada2 > tirada1 && tirada2 >= 7) {  
                j2.sumarPunto();  
            }  
        }  
    }  
}
```

Se le pide al
jugador que tire
los dados

Clase Juego-Ganador

```
Jugador jj = this.getGanador();

if (jj != null) {
    System.out.println("Gano : " + jj.getNombre());
} else {
    System.out.println("Empate");
}
```

```
public Jugador getGanador() {
    if (j1.getPuntos() > j2.getPuntos()) {
        return j1;
    } else {
        if (j2.getPuntos() > j1.getPuntos()) {
            return j2;
        } else {
            return null;
        }
    }
}
```

Otras mejoras en el Juego

--- Constantes en Código

Cuando hay una constante en el código significa que si quiero cambiar ese valor en algún momento, debo modificar el código, o crear una clase nueva.....

```
public void jugar() {  
    int tirada1, tirada2;  
    for (int i = 0 ; i<10; i++) {  
        tirada1 = j1.tirarDados(d1, d2);  
        tirada2 = j2.tirarDados(d1, d2);  
  
        if (tirada1>tirada2 && tirada1<=7) {  
            j1.sumarPunto();  
        } else {  
            if (tirada2>tirada1 && tirada2<=7) {  
                j2.sumarPunto();  
            }  
        }  
    }  
}
```

Incorporar atributos en la clase que me permitan variar el comportamiento, jugar 10 rondas, 20 o 30 o que el mínimo sea 8 o 4

Juego

```
public void jugar() {  
    int tirada1, tirada2;  
    for (int i = 0 ; i < rondas ; i++) {  
        tirada1 = j1.tirarDados(d1, d2);  
        tirada2 = j2.tirarDados(d1, d2);  
  
        if (tirada1 > tirada2 && tirada1 > minimo) {  
            j1.sumarPunto();  
        } else {  
            if (tirada2 > tirada1 && tirada2 > minimo) {  
                j2.sumarPunto();  
            }  
        }  
    }  
}
```

```
public class Juego3 {  
    private Jugador j1;  
    private Jugador j2;  
    private Dado d1;  
    private Dado d2;  
  
    private int rondas;  
    private int minimo;
```

Cubilete

- Tiene un arreglo de dados
- Es responsable de tirar los dados
- Ahora el Juego tiene un cubilete
- Debo modificar el jugador para que reciba un cubilete en lugar de dos dados



Cubilete

Constructores

Get y set

```
public class Cubilete {
```

```
    Dado[] dados;
```

Estado, los dados

```
    public Cubilete() {  
        this(2,6);  
    }
```

```
    public Cubilete(int cantidad, int carasDados) {  
        dados = new Dado[cantidad];  
        for(int i =0; i< dados.length; i++) {  
            dados[i] = new Dado(carasDados);  
        }  
    }
```

```
    public Cubilete(Dado[] nuevosDados) {  
        dados = nuevosDados;  
    }
```

```
    public Dado[] getDados() {  
        return dados;  
    }
```

```
    public void setDados(Dado[] dados) {  
        this.dados = dados;  
    }
```

Cubilete

```
public int tirar() {  
    int suma = 0;  
    for(int i =0; i<dados.length; i++) {  
        suma = suma + dados[i].tirar();  
    }  
    return suma;  
}
```

Debo recorrer el arreglo y a cada objeto (dado) pedirle que se tire, el cubilete es el que suma y retorna ese valor

Juego

Constructores

Atributos

```
public class Juego4 {  
    private Jugador j1;  
    private Jugador j2;  
    private Cubilete vasito;  
  
    private int rondas;  
    private int minimo;  
  
    public Juego4() {  
        this(new Cubilete(2,6));  
    }  
  
    public Juego4(Cubilete vaso) {  
        this(vaso, new Jugador("El 1"), new Jugador("El 2"));  
    }  
  
    public Juego4(Cubilete vaso, Jugador j1, Jugador j2) {  
        this.j1 = j1;  
        this.j2 = j2;  
        vasito = vaso;  
        rondas = 10;  
        minimo = 7;  
    }  
}
```

Juego

— — —

Al jugador se le da el cubilete y es el que tira los dados

```
public void jugar() {
    int tirada1, tirada2;
    for (int i = 0 ; i < rondas; i++) {
        System.out.println("RONDA: " + i);
        tirada1 = j1.tirarDados(vasito);
        tirada2 = j2.tirarDados(vasito);

        if (tirada1 > tirada2 && tirada1 >= minimo) {
            j1.sumarPunto();
        } else {
            if (tirada2 > tirada1 && tirada2 >= minimo) {
                j2.sumarPunto();
            }
        }
    }
    Jugador jj = this.getGanador();
    if (jj != null) {
        System.out.println("Gano : " + jj.getNombre());
    } else {
        System.out.println("Empate");
    }
}
```